



Setembro de 2025

Daniel Ganz Musse, João Vitor de Souza Hernandez, Flavio Augusto da Cruz Melo, Matheus Lowen, Enrico Malho Bozza

Introdução e Objetivos

Objetivos de Negócio Subjacentes

- Criar uma plataforma integrada de delivery de alimentos que conecte clientes, restaurantes, entregadores e locadores de veículos.
- Facilitar a experiência do cliente com usabilidade, variedade de restaurantes, opções de pagamento e acompanhamento em tempo real.
- Oferecer ao restaurante ferramentas para gerenciar pedidos, cardápio e horários de funcionamento.
- Garantir ao entregador meios de realizar entregas eficientes, incluindo suporte para aluguel de veículos.
- Proporcionar ao locador de veículos uma forma prática de disponibilizar e administrar seu inventário.
- Manter um sistema seguro e confiável, garantindo autenticação, registros de atividades e relatórios de desempenho.

Recursos Essenciais

- Catálogo de restaurantes e cardápios navegável.
- Carrinho de compras e sistema de pedidos com opções de pagamento digital.
- Acompanhamento em tempo real do status do pedido e da entrega.
- Notificações push e por e-mail para clientes, restaurantes e entregadores.
- Gestão de veículos para entregadores e locadores (cadastro, aluguel, devolução).
- Painel administrativo para controle de usuários, pedidos, transações e relatórios.

Requisitos Funcionais Essenciais

- Cadastro, login e autenticação de diferentes perfis (RF1, RF25).
- Realização de pedidos e pagamentos (RF3, RF5).

- Acompanhamento de pedidos em tempo real (RF6).
- Avaliação de serviços por clientes (RF7).
- Gerenciamento de cardápio e status de pedidos pelos restaurantes (RF9–RF12).
- Gestão de entregas e rotas pelos entregadores (RF13–RF16).
- Aluguel e devolução de veículos (RF17–RF23).
- Registro de logs, notificações e geração de relatórios (RF26–RF29).

Objetivos de Qualidade para a Arquitetura

- **Segurança:** autenticação, autorização e proteção de dados de usuários e transações.
- **Escalabilidade:** suportar aumento no número de usuários, restaurantes e pedidos.
- **Confiabilidade:** garantir disponibilidade contínua do sistema, principalmente em horários de pico.
- **Desempenho:** tempo de resposta rápido na navegação, pedidos e atualizações de status.
- **Usabilidade:** interface intuitiva para clientes, restaurantes, entregadores e locadores.
- **Manutenibilidade:** arquitetura modular que facilite atualização de funcionalidades.

Partes Interessadas Relevantes e suas Expectativas

- **Clientes:** agilidade na navegação, facilidade de pagamento, segurança e acompanhamento transparente dos pedidos.
- **Restaurantes:** ferramentas eficazes para gerenciar pedidos, cardápios e horários.
- **Entregadores:** flexibilidade na aceitação de entregas, rotas otimizadas e opções de veículos.
- **Locadores de veículos:** controle de inventário, definição de preços e histórico de transações.
- **Equipe de administração do sistema:** controle sobre usuários, monitoramento de operações e relatórios de desempenho.

Visão Geral dos Requisitos

Conteúdo

Este documento oferece uma descrição concisa dos requisitos funcionais para a ClickDelivery, uma plataforma integrada de delivery de alimentos e aluguel de veículos para entregadores. As forças motrizes do sistema são a otimização da experiência do cliente ao pedir comida, a facilitação da gestão de pedidos para os restaurantes e a capacitação dos entregadores com acesso facilitado a veículos, conectando-os também a locadores.

O sistema abrangerá funcionalidades para cinco perfis principais:

Cliente: Cadastro, login, navegação em restaurantes, realização e acompanhamento de pedidos, pagamento e avaliação.

Restaurante: Gerenciamento de cardápio, recebimento e atualização de status de pedidos e controle do horário de funcionamento.

Entregador: Aceite de entregas, acompanhamento de rotas, e um sistema completo para alugar, renovar e devolver veículos.

Locador de Veículos: Cadastro e gestão de inventário de veículos, definição de preços e acompanhamento de aluguéis.

Administração do Sistema: Gerenciamento de usuários, garantia de segurança, processamento de pagamentos, envio de notificações e geração de relatórios.

Motivação

Do ponto de vista dos usuários finais, o sistema é criado para unificar e aprimorar a cadeia de delivery de alimentos. Para os clientes, busca-se oferecer uma experiência de pedido mais fluida, transparente e confiável. Para os restaurantes, a meta é simplificar a gestão de pedidos e cardápios, ampliando seu alcance de mercado. Para os entregadores, o sistema não só oferece uma fonte de renda, mas também remove uma barreira significativa ao facilitar o acesso a veículos por meio de um sistema de aluguel integrado. Para os locadores, abre-se um novo nicho de mercado. O objetivo final é melhorar a qualidade e a eficiência de toda a atividade comercial de delivery.

Forma

A descrição dos requisitos funcionais pode ser resumida em formato tabular de casos de uso, conforme o exemplo abaixo. Esta visão geral serve como um guia rápido, e para detalhes aprofundados, os documentos de especificação de requisitos devem ser consultados.

Ator	Caso de Uso Principal	Requisitos Funcionais Associados
Cliente	Realizar um pedido de comida	RF1, RF2, RF3, RF4, RF5, RF6, RF8
	Avaliar o serviço	RF7
Restaurante	Gerenciar pedidos e cardápio	RF9, RF10, RF11, RF12
Entregador	Realizar uma entrega	RF13, RF14, RF15, RF16
	Alugar um veículo	RF17, RF18, RF19, RF20
Locador de Veículos	Gerenciar frota para aluguel	RF21, RF22, RF23, RF24
Administrador	Gerenciar a plataforma	RF25, RF26, RF27, RF28, RF29, RF30

Objetivos de Qualidade

A seguir estão os três objetivos de qualidade mais críticos para a arquitetura do sistema, em ordem de prioridade. A seleção foi baseada na análise dos requisitos funcionais que definem o núcleo do negócio e as interações entre os diferentes tipos de usuários: Clientes, Restaurantes, Entregadores e Locadores de Veículos.

Prioridade	Objetivo de Qualidade (ISO 25010)	Cenário Concreto de Qualidade
1	Usabilidade	Um novo entregador, com treinamento mínimo, deve ser capaz de se cadastrar, encontrar e alugar um veículo, e aceitar sua primeira entrega em menos de 15 minutos após abrir o aplicativo pela primeira vez. A interface deve ser clara e o fluxo de ações, intuitivo, para minimizar a necessidade de suporte.
2	Confiabilidade	Durante o horário de pico (19h às 21h de sexta-feira), 99.9% das notificações de novos pedidos devem ser entregues aos restaurantes e entregadores em menos de 5 segundos após a confirmação do cliente. O sistema deve manter a capacidade de processar o rastreamento de status do pedido em tempo real sem falhas.
3	Segurança	As informações de pagamento do cliente (ex: cartão de crédito) devem ser processadas através de um gateway de pagamento certificado, sem que dados

		sensíveis sejam armazenados de forma legível no sistema. O sistema deve garantir a correta autenticação e autorização para que um usuário de um perfil (ex: Entregador) não possa acessar funcionalidades exclusivas de outro (ex: Locador de Veículos)
--	--	---

Partes Interessadas

Conteúdo

Esta análise identifica os grupos de interesse (stakeholders) cruciais para o sucesso da plataforma de delivery, que integra também o aluguel de veículos. Cada um destes grupos possui um papel ativo e expectativas específicas sobre a arquitetura do sistema. Eles incluem:

Clientes Finais: A base de usuários que realiza os pedidos. Eles precisam de uma plataforma rápida, confiável e segura.

Restaurantes Parceiros: Utilizam o sistema para gerenciar cardápios, receber e preparar pedidos. A estabilidade da plataforma impacta diretamente seu faturamento.

Entregadores: A força de trabalho logística que utiliza o aplicativo para aceitar entregas e, crucialmente, para alugar veículos. Precisam de eficiência, boa usabilidade e informações precisas.

Locadores de Veículos: Parceiros de negócio que gerenciam o inventário de veículos disponíveis para os entregadores. Necessitam de um sistema robusto para controle financeiro e de frota.

Equipe de Administração e Desenvolvimento: O time interno que irá projetar, construir, manter e tomar decisões técnicas sobre a arquitetura e a evolução do sistema.

Motivação

É vital conhecer as necessidades de todos os envolvidos, clientes, restaurantes, entregadores e locadores de veículos, para evitar surpresas que possam comprometer o projeto.

Ignorar as expectativas de qualquer um desses grupos resultará em falhas críticas: um sistema lento afastará clientes, notificações instáveis causarão perda de pedidos para os restaurantes, e problemas na logística ou no aluguel de veículos levarão a entregas atrasadas. Essencialmente, as necessidades de cada stakeholder definem os requisitos técnicos e o sucesso da arquitetura do sistema.

Forma

Função/Nome	Contato	Expectativas
Cliente	Notificações Push (status do pedido, promoções), E-mail (cadastro, recibos), Interface do App (mensagens e alertas).	Espera uma arquitetura que garanta alta disponibilidade e desempenho, especialmente em horários de pico. Segurança robusta para dados pessoais e de pagamento. O sistema deve ser intuitivo, rápido e fornecer atualizações de status em tempo real de forma confiável.
Restaurante	Painel do Restaurante (dashboard web/app), Alertas Sonoros para novos pedidos, Notificações Push.	Precisa de uma plataforma estável e confiável para gerenciamento de cardápios e pedidos. O sistema de notificação de novos pedidos deve ser imediato e à prova de falhas. A arquitetura deve suportar uma interface de gestão simples e eficiente.
Entregador	Notificações Push no aplicativo (novas entregas, atualizações), Interface do Mapa (rotas), SMS (como backup para notificações críticas).	Demanda um aplicativo móvel rápido, com baixo consumo de bateria e dados. A arquitetura deve suportar notificações em tempo real, geolocalização precisa e otimização de rotas. O módulo de aluguel de veículos precisa ser integrado e funcional.
Locador de Veículos	Painel do Locador (dashboard web para gestão de frota e finanças), E-mail (relatórios, notificações de aluguel/devolução).	Espera um portal de gerenciamento seguro para cadastrar veículos e definir preços. A documentação da arquitetura deve detalhar como os dados de aluguéis, pagamentos e cobranças são processados e protegidos.
Administrador do Sistema / Equipe de TI	Dashboards de Monitoramento (logs, métricas), Alertas automáticos (via e-mail, Slack, etc.) sobre falhas ou anomalias, Relatórios Gerenciais.	Precisa de uma documentação de arquitetura completa, detalhando os componentes, APIs e fluxos de dados. A arquitetura deve ser modular, escalável, segura e de fácil manutenção. Exige clareza sobre as estratégias de logging, monitoramento e autenticação.

Restrições Arquiteturais

Esta seção descreve as decisões e condições que limitam a liberdade da equipe de arquitetura e desenvolvimento. Elas são fundamentais para garantir que o produto final atenda a requisitos técnicos, de negócio e regulatórios.

Restrições Técnicas

Restrição	Explicação / Justificativa
Plataforma Exclusivamente Web (Responsiva)	Por decisão de negócio, o sistema não terá aplicativos móveis nativos. Toda a interação dos usuários será via navegador web. Isso exige que a arquitetura seja baseada em tecnologias web e que o design da interface seja totalmente responsivo para garantir a usabilidade em desktops, tablets e smartphones.
Dependência de Serviços de Roteirização e Mapas	O sistema precisa fornecer rotas otimizadas para os entregadores. Isso impõe a integração com uma API de terceiros (ex: Google Maps, Mapbox), sujeitando a arquitetura aos custos, limites de uso e termos de serviço do provedor escolhido.
Integração Obrigatória com Gateway de Pagamento	Para processar pagamentos com cartão de crédito, débito e outras formas digitais, o sistema deve se integrar a um ou mais gateways de pagamento. Isso cria uma dependência externa e exige que a arquitetura siga os padrões de segurança (como PCI DSS) e os fluxos de API definidos pelo provedor.

Restrições Organizacionais e de Negócio

Restrição	Explicação / Justificativa
Conformidade com a Lei Geral de Proteção de Dados (LGPD)	O sistema irá coletar e processar dados pessoais de clientes, entregadores e funcionários de restaurantes, como nome, endereço e informações de contato. A arquitetura deve, obrigatoriamente, seguir os princípios da LGPD.
Idioma Principal: Português do Brasil	Todo o conteúdo e interfaces de usuário devem ser primariamente em Português do Brasil, conforme indicado nos requisitos

	funcionais. A arquitetura deve ser flexível para futura internacionalização.
Modelo de Negócio Baseado em Múltiplos Atores	A arquitetura deve suportar transações financeiras e lógicas complexas entre múltiplos atores (Cliente, Restaurante, Entregador, Locador de Veículos). Isso restringe o design dos módulos de pagamento, comissões e contabilidade.

Contexto e Escopo

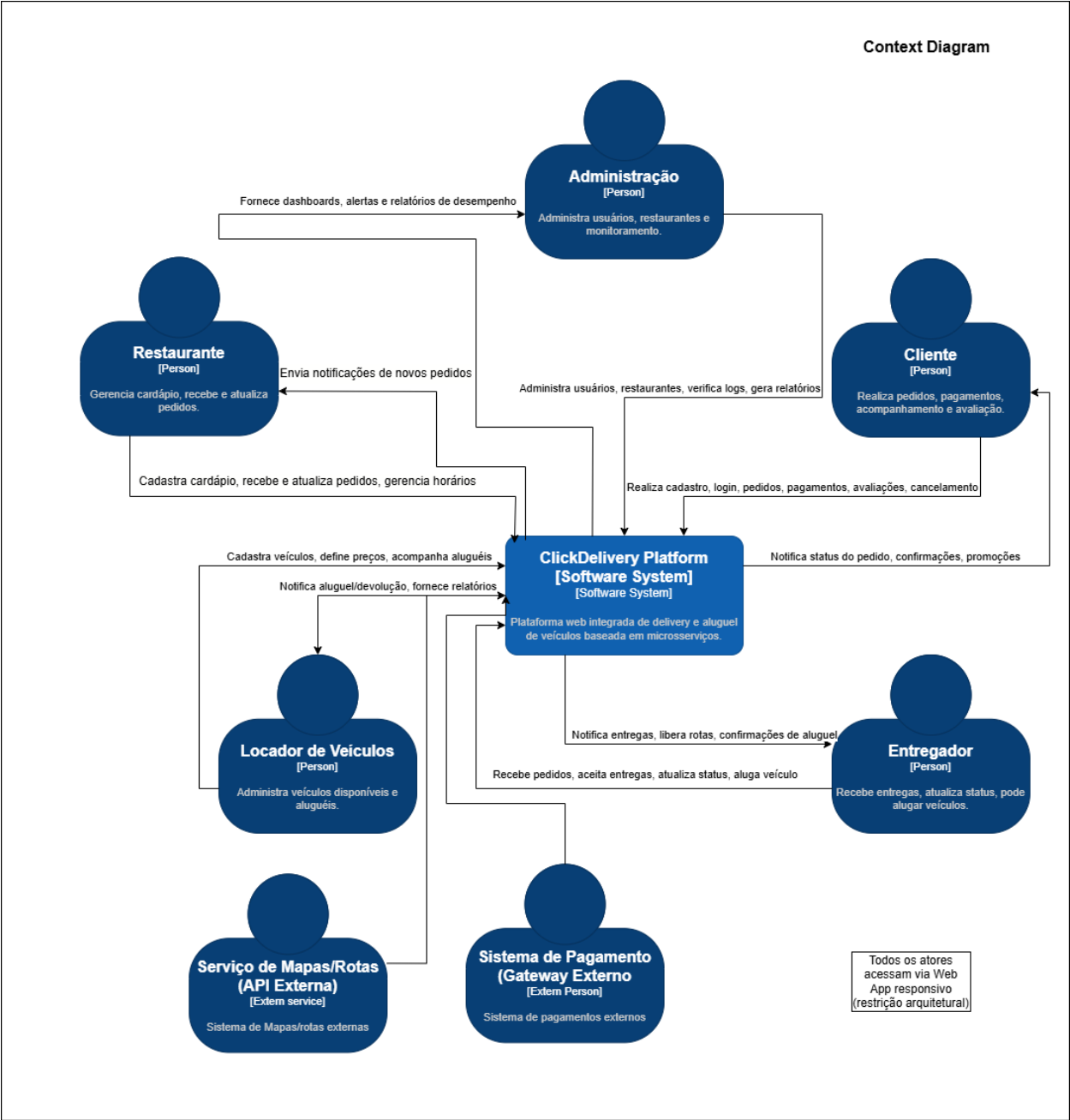
O sistema é uma plataforma de delivery integrada que conecta clientes, restaurantes, entregadores e locadores de veículos.

Entradas de negócio:

- Cadastro/login de usuários (clientes, restaurantes, entregadores, locadores).
- Pedidos de clientes (itens de cardápio, endereço, pagamento).
- Atualização de cardápio e horários (restaurantes).
- Solicitação de entrega e atualização de status (entregadores).
- Cadastro e aluguel de veículos (locadores e entregadores).

Saídas de negócio:

- Confirmação de pedidos e pagamentos.
- Atualização em tempo real do status (pedido e entrega).
- Notificações para clientes, restaurantes, entregadores e locadores.
- Relatórios de desempenho e histórico de pedidos/aluguéis.



Contexto Negocial

O sistema, aqui chamado de "Plataforma de Delivery e Aluguel", atua como um intermediário digital que conecta múltiplos atores. A tabela abaixo descreve os parceiros de comunicação, os dados que eles enviam para o sistema (entradas) e os dados que recebem do sistema (saídas).

Parceiro de Comunicação	Entradas para o Sistema (Inputs)	Saídas do Sistema (Outputs)
-------------------------	----------------------------------	-----------------------------

Cliente	Dados de cadastro e login Seleção de restaurantes e itens de cardápio Endereço de entrega Dados do pedido para o carrinho Informações de pagamento Avaliações sobre restaurantes e entregadores Solicitação de cancelamento do pedido	Confirmação de cadastro/login Catálogo de restaurantes e seus cardápios Confirmação do pedido realizado Status do pedido em tempo real (preparação, em rota, etc.) Notificações gerais (push, email)
Restaurante	Dados para cadastro e atualização do cardápio Atualizações de status do pedido (ex: preparando, pronto para entrega) Configuração de horários de funcionamento	Notificações de novos pedidos Relatórios de desempenho
Entregador	Aceite ou recusa de uma entrega Atualizações de status da entrega (ex: coletado, entregue) Seleção do veículo para aluguel e período desejado Solicitação para renovar ou encerrar o aluguel Confirmação da devolução do veículo	Notificações de pedidos disponíveis para entrega Rotas otimizadas para a entrega Catálogo de veículos disponíveis para aluguel
Locador de Veículos	Dados de cadastro do inventário de veículos (motos, bicicletas) Definição de preços e condições de aluguel para os veículos	Notificações quando um veículo é alugado ou devolvido Histórico de aluguéis e pagamentos
Administrador do Sistema	Comandos para gerenciar usuários e restaurantes (bloqueio, remoção)	Logs de pedidos, transações e entregas Relatórios de desempenho geral da plataforma
Sistema de Pagamento (Gateway Externo)	Confirmação ou recusa da transação de pagamento	Dados da transação para autorização (valor, informações do pedido)

Sistema de Mapas e Rotas (API Externa)	Dados de rotas otimizadas, distâncias e tempos de percurso	Coordenadas de origem (restaurante) e destino (cliente) para cálculo da rota
--	--	--

Contexto Técnico

Interfaces Técnicas (canais e mídias de transmissão)

O sistema de delivery se conecta ao ambiente externo por meio dos seguintes canais técnicos:

- Aplicativo web: canal de acesso para restaurantes e locadores de veículos.
- Painel administrativo (web): canal exclusivo da equipe de administração.
- APIs externas de pagamento: integração com gateways (cartão de crédito, débito, carteiras digitais).
- APIs de mapas e geolocalização: integração para rotas otimizadas e status em tempo real.
- Serviço de notificações push/e-mail/SMS: comunicação em tempo real com clientes, restaurantes, entregadores e locadores.

Mapeamento de Entrada/Saída para Canais

Canal Técnico	Entrada (do usuário/sistema)	Saída (resposta do sistema)
Aplicativo web (restaurante)	Cadastro/atualização de cardápio, horários de funcionamento, atualização de status do pedido	Notificações de pedidos recebidos, relatórios de desempenho
Aplicativo web (locador de veículos)	Cadastro de veículos, definição de preços, consulta de histórico	Notificações de aluguel/devolução, relatórios de histórico
Painel administrativo (web)	Gestão de usuários, bloqueio/remoção de contas, geração de relatórios	Relatórios de desempenho, logs de pedidos e transações
APIs externas de pagamento	Dados de transação (pedido, aluguel, cartão/digital wallet)	Confirmação/recusa de pagamento

APIs de mapas/geolocalização	Solicitação de rota e posição do entregador	Rotas otimizadas, estimativas de tempo
Notificações push/e-mail/SMS	Disparo de eventos (pedido confirmado, status atualizado, entrega concluída)	Recebimento da mensagem pelo usuário final

Motivação

- As interfaces técnicas definem como o sistema se comunica com usuários e serviços externos, sendo fundamentais para desempenho, escalabilidade e segurança.
- Stakeholders técnicos (como arquitetos de infraestrutura e desenvolvedores backend) tomam decisões com base nesses canais, escolhendo protocolos, APIs, protocolos de rede e servidores.
- A clareza no mapeamento de entrada/saída garante integração consistente, evita falhas de comunicação e permite auditoria e rastreabilidade.

Estratégia de Solução

Conteúdo

Esta seção apresenta um resumo das decisões fundamentais que moldam a arquitetura do sistema de delivery e aluguel de veículos. O objetivo é criar uma plataforma robusta, escalável e de fácil manutenção, capaz de atender às necessidades de clientes, restaurantes, entregadores e locadores de veículos.

As decisões-chave incluem:

Decisões de tecnologia: Adoção de uma stack de tecnologia moderna, baseada em nuvem e orientada a serviços, para garantir performance e escalabilidade.

Decisões sobre a decomposição de nível superior do sistema: A escolha de um padrão de arquitetura de microsserviços para decompor o sistema em domínios de negócio independentes, como Pedidos, Usuários, Entregas e Aluguel de Veículos.

Decisões sobre como atingir as principais metas de qualidade: Estratégias para garantir alta disponibilidade, segurança nas transações, performance em tempo real e manutenibilidade do sistema.

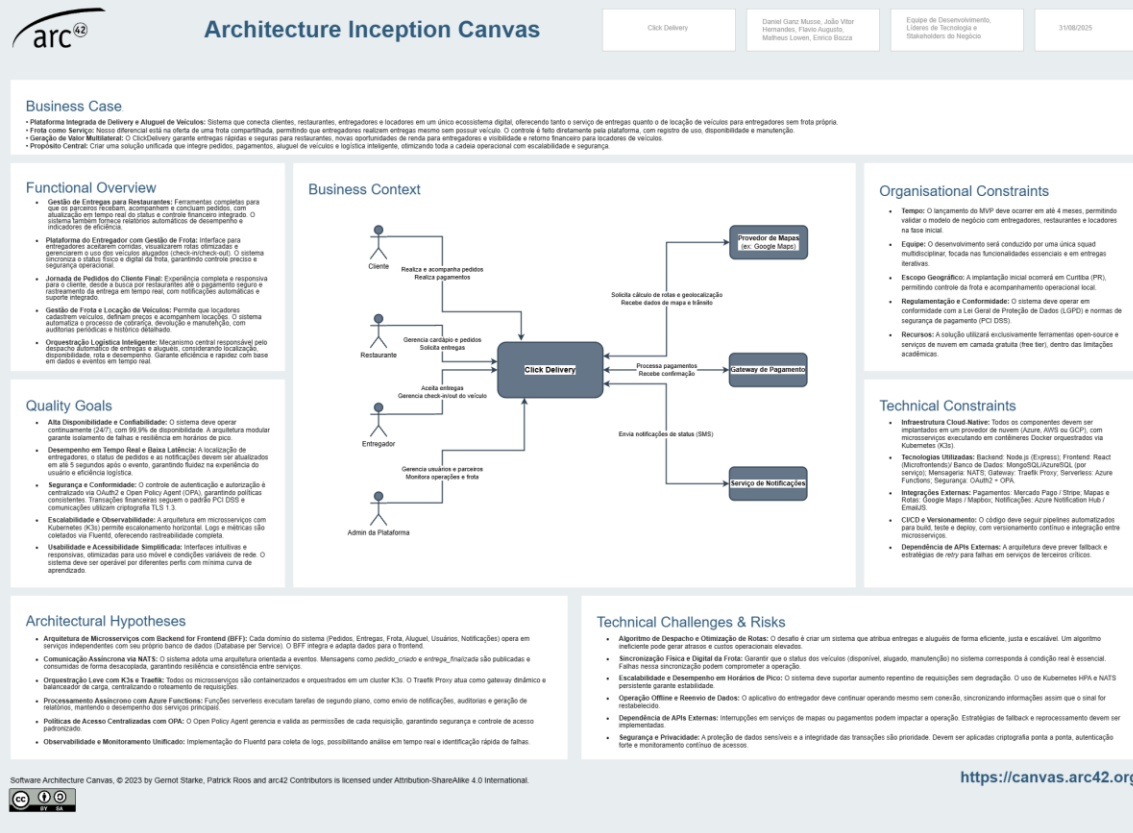
Decisões organizacionais relevantes: Utilização de serviços de terceiros para funcionalidades especializadas (pagamentos, mapas) e adoção de uma metodologia de desenvolvimento ágil.

Motivação

As decisões a seguir formam os pilares da arquitetura. Elas foram tomadas para endereçar a complexidade de um sistema com múltiplos atores e domínios de negócio distintos (delivery de comida e aluguel de veículos). A escolha por microsserviços, por exemplo, é motivada pela necessidade de escalar e manter cada parte do sistema de forma independente, evitando que uma falha no sistema de aluguel afete o processamento de pedidos de comida.

Forma

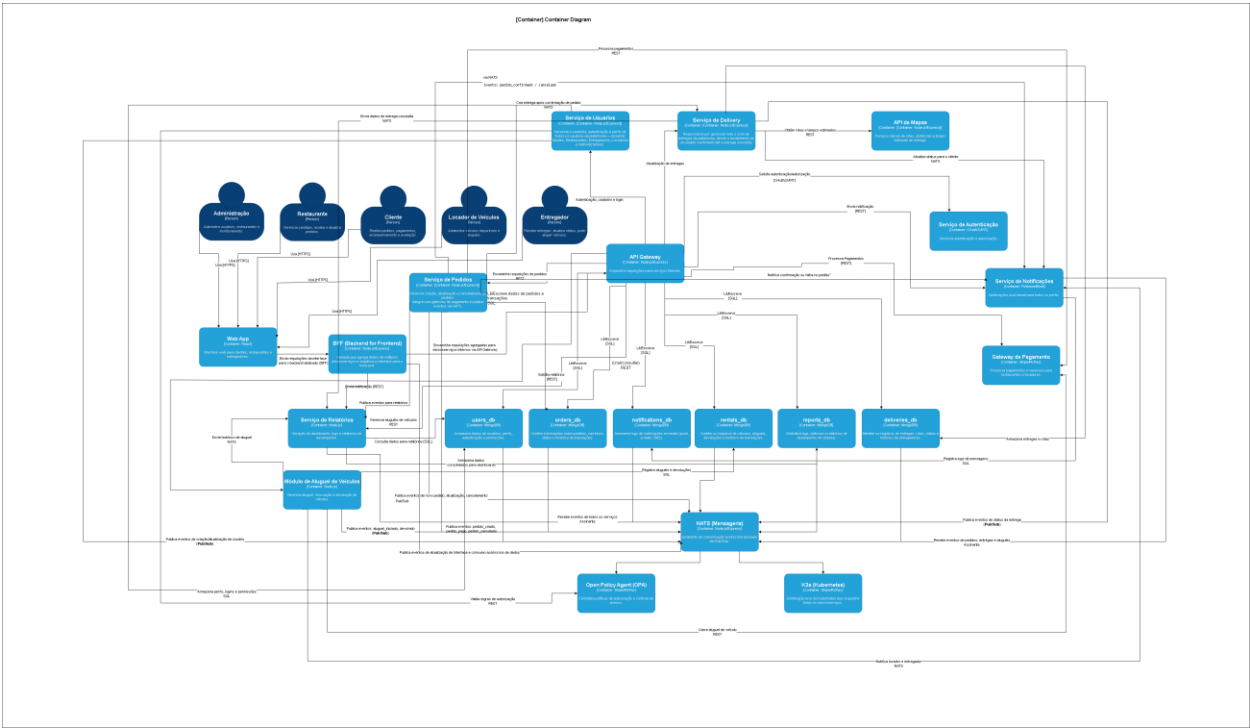
Decisão Chave	O Que Foi Decidido	Motivação / Justificativa
Arquitetura do Sistema	Adoção do padrão de microsserviços para separar os domínios de negócio (Pedidos, Aluguel, etc.).	Facilitar a manutenção, permitir escalabilidade independente de cada módulo e isolar falhas.
Tecnologia e Infraestrutura	Plataforma Cloud-Native (Nuvem, Contêineres) com React.	Garantir escalabilidade, agilidade no desenvolvimento e reuso de código.
Metas de Qualidade	Foco em Segurança (com gateways de pagamento) e Tempo Real (com WebSockets).	Proteger dados financeiros sensíveis e atender aos requisitos de rastreamento ao vivo.
Delegação a Terceiros	Uso de APIs externas para funcionalidades de Mapas, Pagamentos e Notificações.	Acelerar o desenvolvimento e focar na lógica de negócio principal, usando soluções especializadas.



Visão de Blocos de Construção

A arquitetura segue o modelo de **microserviços**. No **Nível 1 (caixa preta)** temos os blocos principais:

Bloco	Responsabilidade
Módulo de Usuários	Cadastro, login, autenticação e perfis (Cliente, Restaurante, Entregador, Locador, Admin).
Módulo de Pedidos	Carrinho, criação de pedidos, integração com pagamento, status em tempo real.
Módulo de Restaurantes	Controle de cardápio, horários, atualização do status dos pedidos.
Módulo de Entregas	Aceite/recusa de entregas, rastreamento em rota, atualização de status.
Módulo de Aluguel de Veículos	Cadastro de veículos, aluguel, devolução e histórico de transações.
Módulo de Notificações	Envio de push, e-mails e alertas.
Módulo Administrativo	Gestão de usuários, relatórios, logs e auditoria.
Integrações Externas	Gateways de pagamento, APIs de mapas/rotas, serviços de notificações.



Visão Sistêmica Geral de Caixa Branca

Nível 1 – Caixa Branca (visão geral do sistema)

O sistema é composto pelos blocos acima (cada um um microsserviço) + um **API Gateway** para orquestração. Comunicação assíncrona é feita via mensageria (NATS pub/sub).

- Interface principal: **Web App responsivo** (não há app nativo, por restrição arquitetural).
- Serviços críticos: **Pedidos, Notificações, Pagamentos e Entregas.**

Nível 2 – Estrutura interna de alguns blocos

Exemplo do **Módulo de Pedidos** (mais crítico):

- **OrderController** → recebe requisições do usuário.
- **OrderService** → processa lógica de pedidos.
- **PaymentService** (chamando gateway externo).
- **TrackingService** → envia atualizações de status para o cliente.
- **EventPublisher** → publica eventos em NATS (para notificações, entregas, relatórios).

Outro exemplo: **Módulo de Aluguel de Veículos:**

- **VehicleRentalController**
- **VehicleInventoryService**
- **RentalTransactionService**
- **IntegrationPaymentAdapter** (para aluguel)

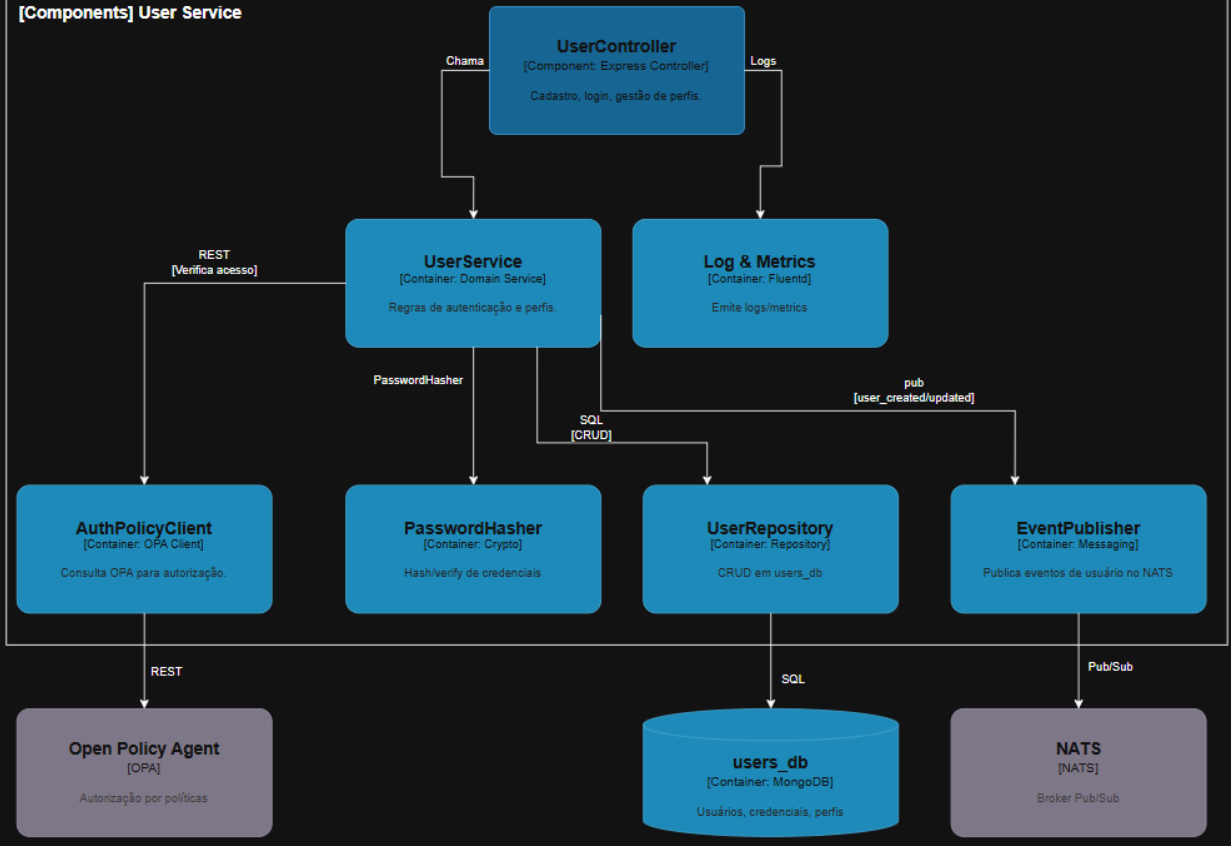
Nível 3 – Detalhamento interno de serviços específicos

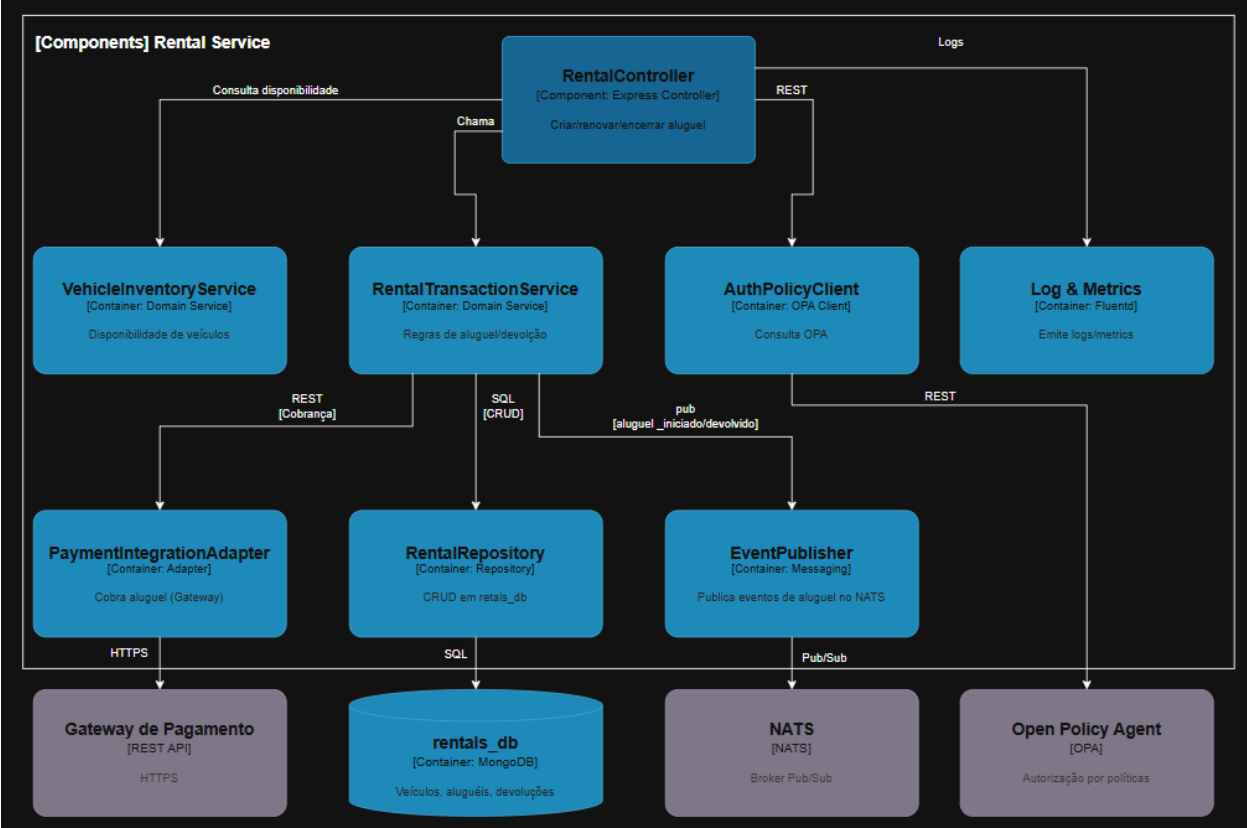
Exemplo: **PaymentService**

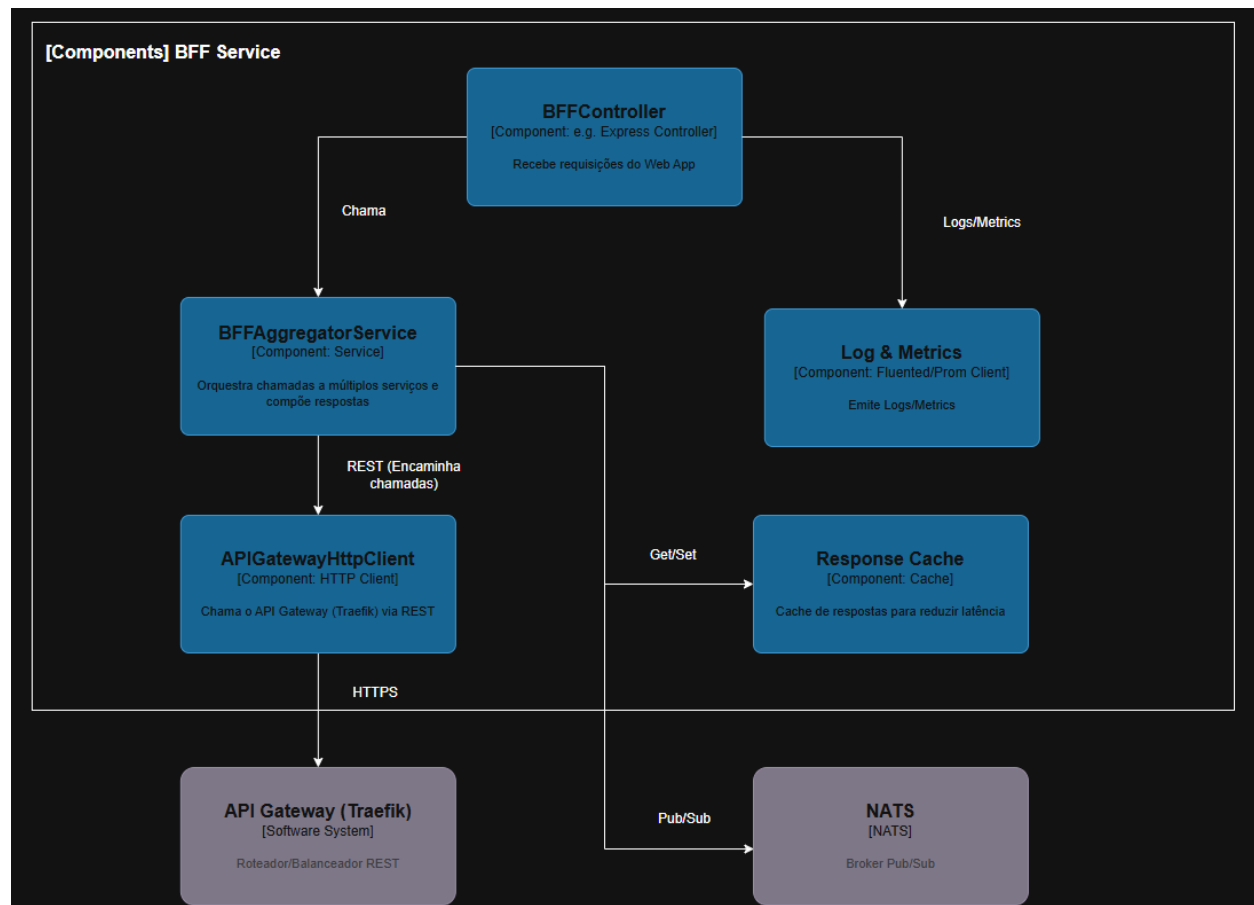
- **PaymentProcessor** (orquestra chamadas ao gateway externo).
- **PaymentValidator** (verifica dados e antifraude).
- **PaymentRepository** (salva logs de transação).
- **IntegrationAdapter** (abstrai o gateway para trocar de provedor sem impacto).



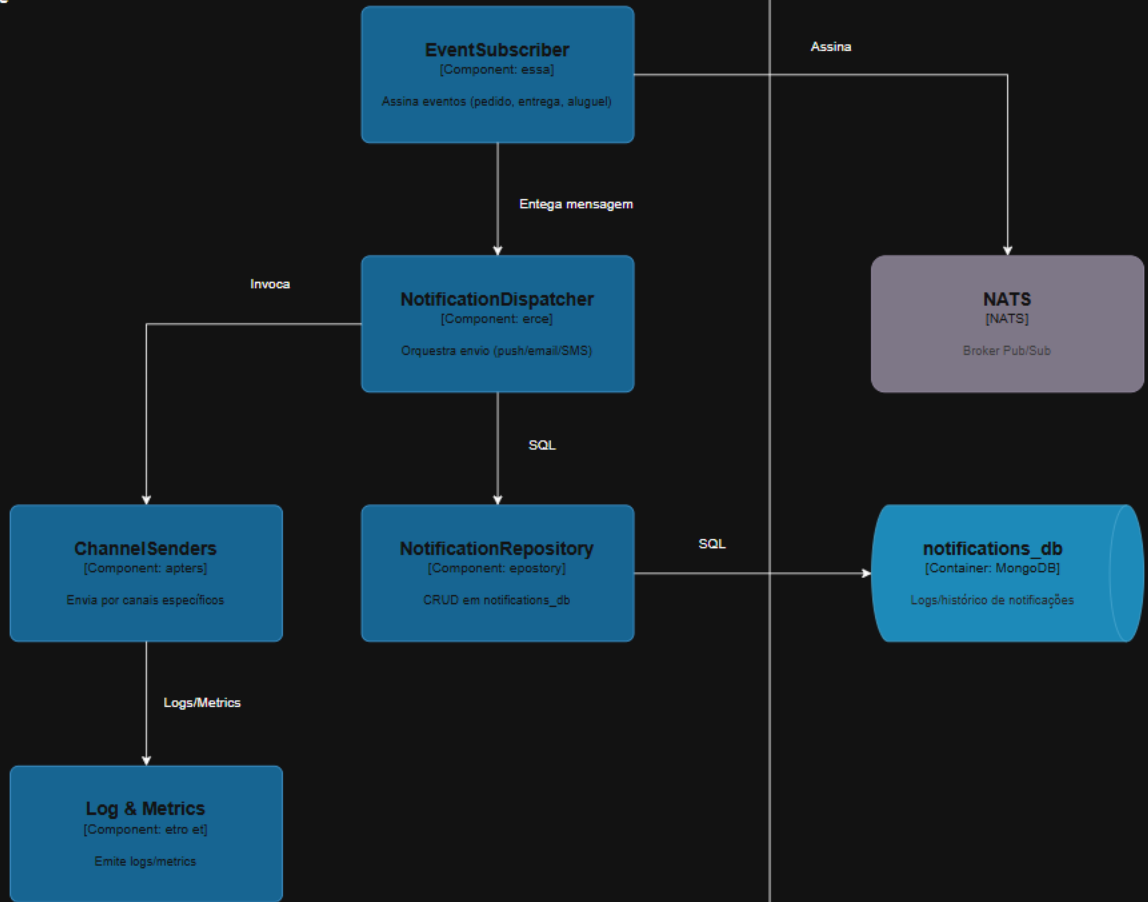
[Components] User Service

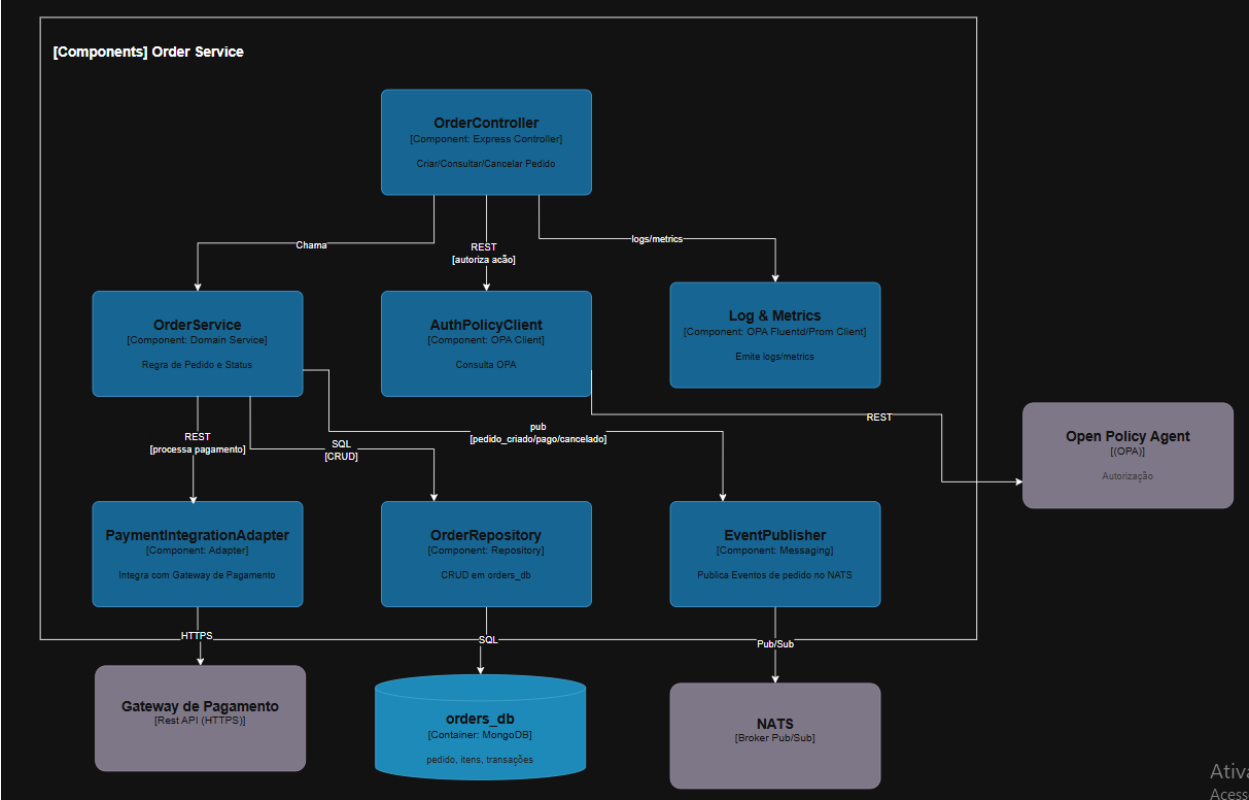
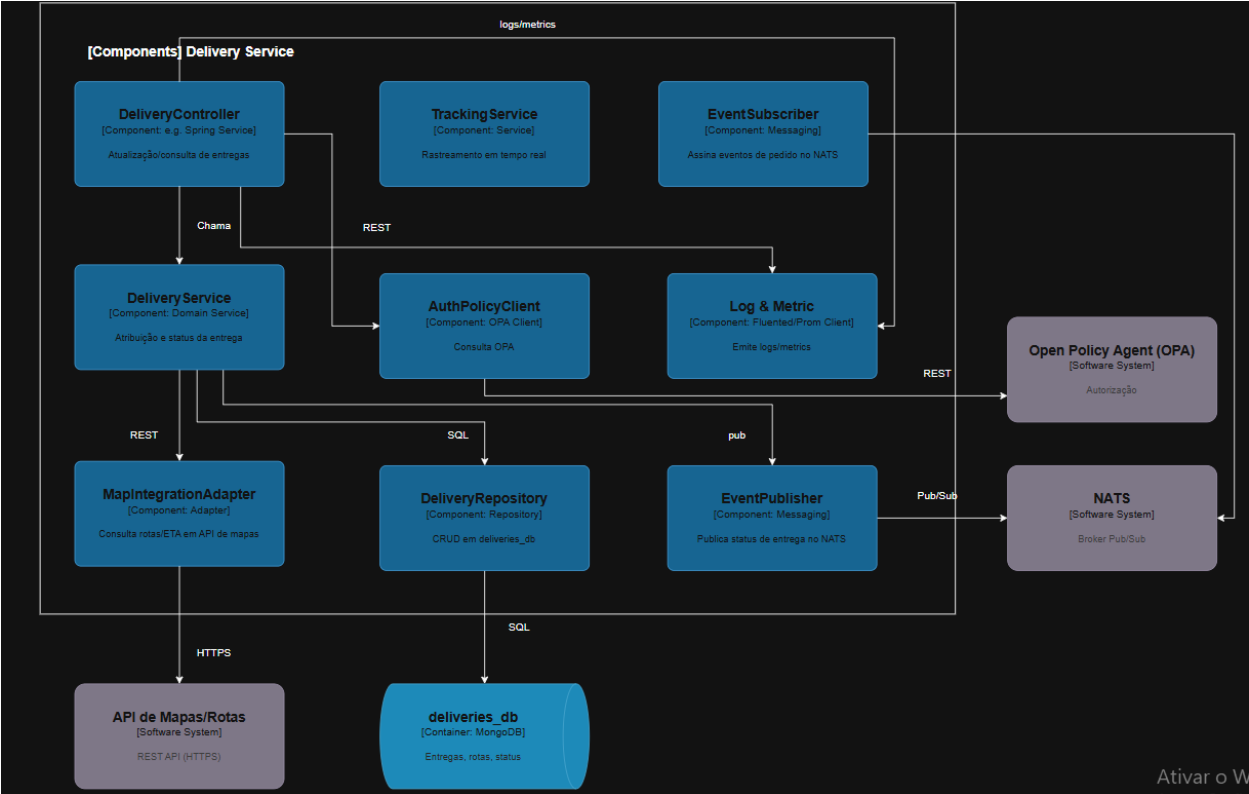






[Components] Notification Service

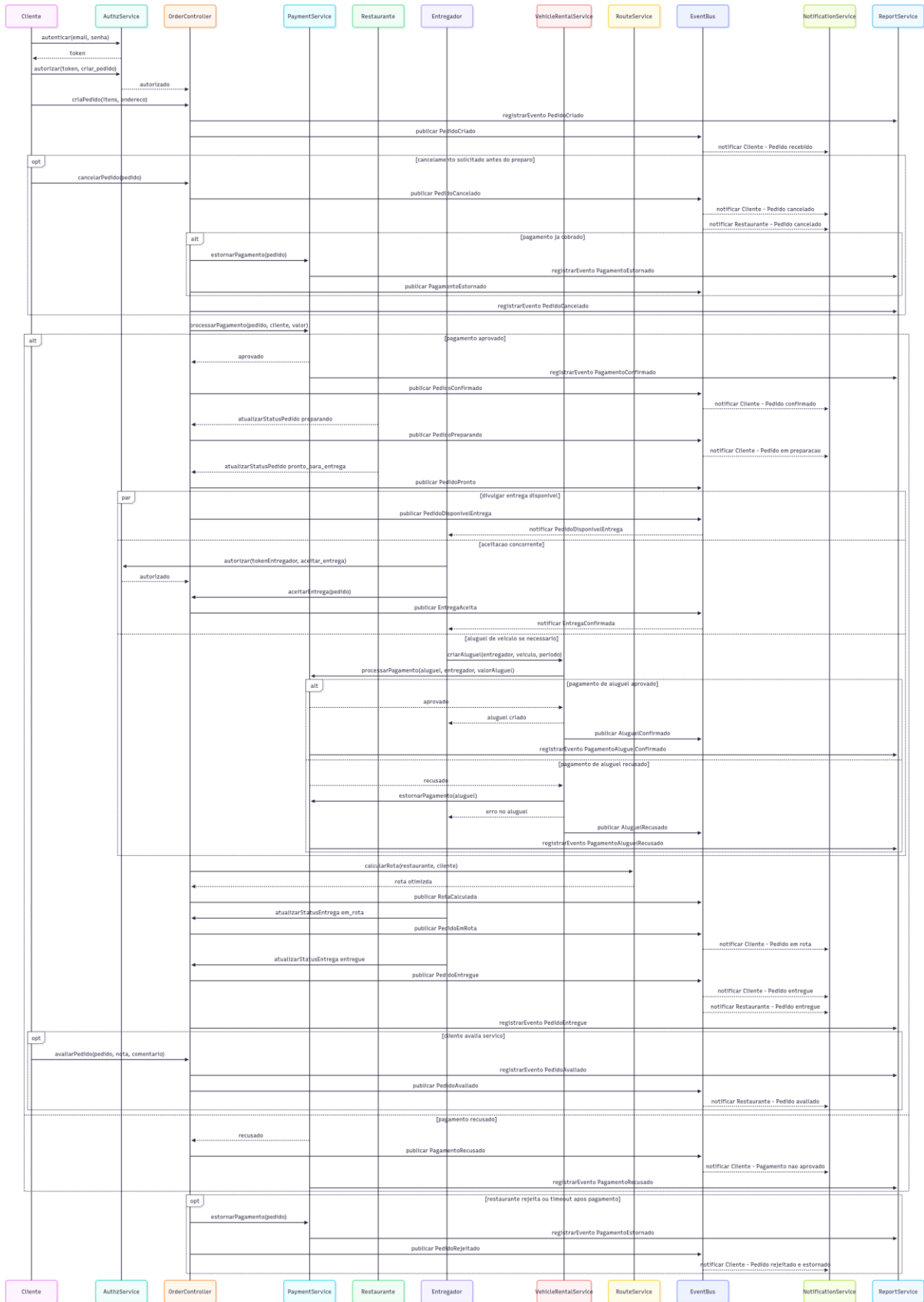




- a. **Módulo de Restaurantes** recebe notificação de novo pedido.
 - b. **Módulo de Entregas** publica evento para entregadores disponíveis.
 - c. **Módulo de Notificações** envia push/email para todos os envolvidos.
4. Entregador aceita → API de Mapas calcula rota.
5. Cliente acompanha status em tempo real (via **TrackingService**).
6. Entrega finalizada → **Módulo Administrativo** registra log + relatório.

Cenário de Exceção: Falha no Pagamento

- Gateway retorna recusa → **Módulo de Pedidos** cancela o pedido.
- **Módulo de Notificações** avisa imediatamente o cliente.
- **Módulo Administrativo** grava log do erro.



Visão de Implantação

Conteúdo

A implantação do sistema é feita em um provedor de nuvem (Cloud-Native), utilizando uma arquitetura de microsserviços em contêineres. A orquestração é gerenciada por Kubernetes (K3s). A infraestrutura conta com um API Gateway (Traefik Proxy) como ponto de entrada para as requisições, um sistema de mensageria (NATS) para comunicação assíncrona e notificações em tempo real (atendendo a

RF6, RF27), e um banco de dados gerenciado. Cada domínio funcional (Pedidos, Aluguel etc.) é um serviço independente, e o sistema se integra a APIs externas para Pagamentos e Mapas.

Motivação

As decisões de implantação foram tomadas para atender aos principais requisitos de qualidade do sistema:

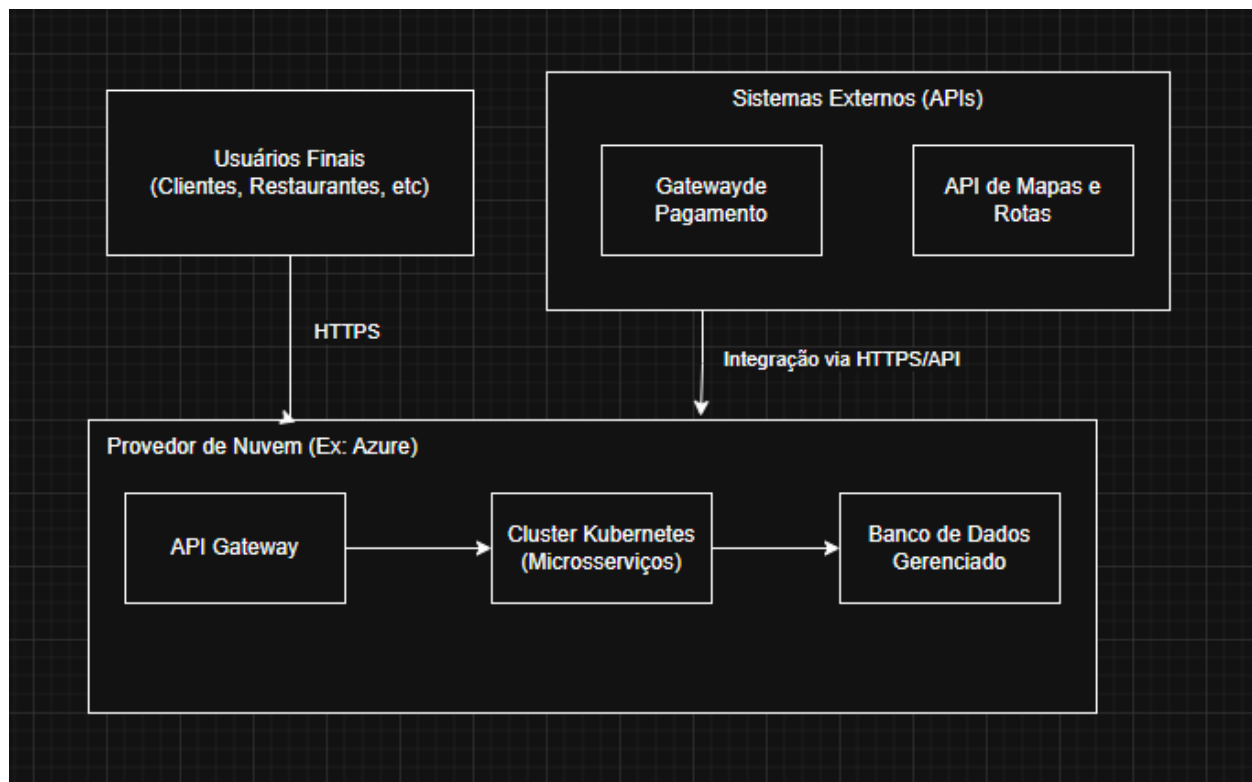
Escalabilidade: A arquitetura com Kubernetes foi escolhida para suportar o crescimento no volume de usuários e transações (RF1, RF3, RF29) sem degradar a performance, permitindo que cada serviço escale de forma independente.

Confiabilidade: Para garantir a operação contínua do negócio, que depende de notificações críticas (RF14, RF27), a arquitetura isola falhas entre os serviços e utiliza mensageria (NATS) para a entrega garantida de eventos.

Segurança: Para proteger os dados sensíveis de usuários e pagamentos (RF5), a implantação utiliza um ambiente de nuvem seguro e um controle de acesso centralizado (OPA), cumprindo o requisito de autorização RF25.

Nível de Infraestrutura 1

Esta seção descreve a distribuição de alto nível do sistema, suas justificativas, características de qualidade e o mapeamento dos componentes de software para a infraestrutura.



Motivação

A estrutura de implantação foi motivada pela necessidade de criar uma plataforma robusta que atenda aos múltiplos atores do sistema (clientes, restaurantes, entregadores, locadores). A adoção de uma abordagem Cloud-Native com um cluster Kubernetes (K3s) foi uma decisão estratégica para garantir a escalabilidade e a resiliência do sistema. Essa arquitetura permite que cada microserviço (como Pedidos ou Aluguel de Veículos) seja escalado de forma independente para suportar picos de demanda, como horários de alta movimentação de pedidos, atendendo diretamente ao requisito de escalabilidade. A separação do banco de dados em um serviço gerenciado aumenta a durabilidade e a confiabilidade dos dados transacionais, que são críticos para o negócio.

Características de Qualidade e/ou Desempenho

A infraestrutura foi projetada para suportar os objetivos de qualidade mais críticos definidos para o sistema:

Confiabilidade: A implantação em um cluster orquestrado garante alta disponibilidade (meta de 99.9%) , assegurando que o sistema permaneça operacional e que as notificações de novos pedidos sejam entregues de forma garantida.

Escalabilidade: O ambiente de nuvem e o Kubernetes permitem a expansão horizontal dos serviços para lidar com o aumento de usuários, restaurantes e pedidos sem comprometer o desempenho. Isso é fundamental para suportar o crescimento do negócio para novas cidades.

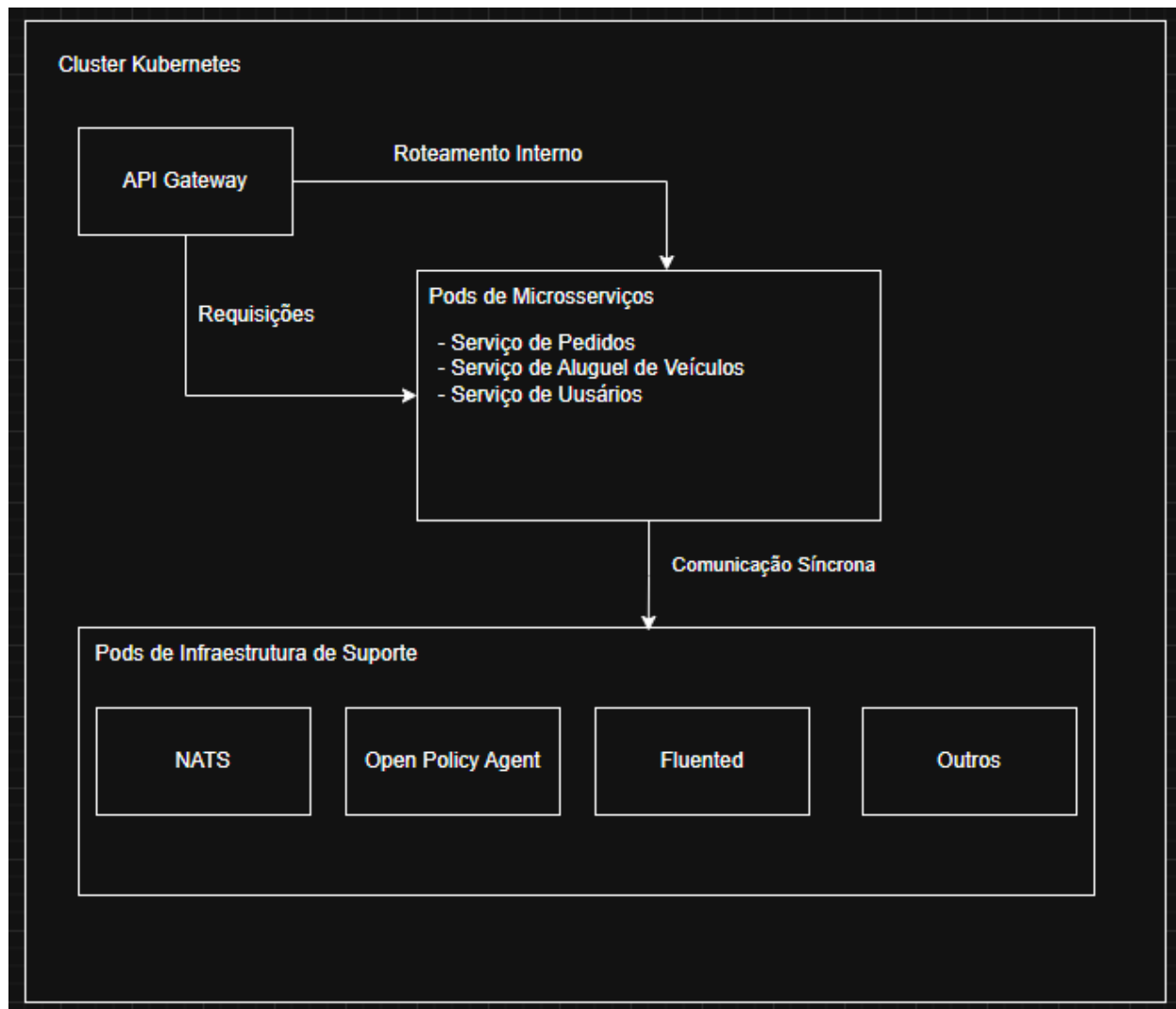
Desempenho: A arquitetura distribuída é otimizada para fornecer tempos de resposta rápidos para operações críticas, como a confirmação de um pedido (< 2 segundos) e a atualização de status em tempo real.

Mapeamento de Blocos de Construção para Infraestrutura

O mapeamento dos artefatos de software para os elementos da infraestrutura é detalhado na tabela abaixo:

Bloco de Construção (Software)	Elemento de Infraestrutura (Hardware/Plataforma)
Microserviços (Módulo de Pedidos, Aluguel, etc.)	Contêineres executados como Pods no Cluster Kubernetes (K3s).
API Gateway (Traefik Proxy)	Contêiner/Pod especializado no Cluster Kubernetes, atuando como Ingress Controller.
Sistema de Mensageria (NATS)	Contêiner/Pod especializado implantado no Cluster Kubernetes.
Aplicação Web (Frontend)	Serviço de hospedagem de arquivos estáticos na nuvem ou contêiner web, conforme a restrição de ser uma plataforma exclusivamente web.
Dados da Aplicação (Pedidos, Transações, Usuários)	Banco de Dados Gerenciado (ex: AWS RDS, Google Cloud SQL), externo ao cluster de computação.

Nível de Infraestrutura 2



- **Pods dos Microserviços de Aplicação:** Cada bloco de construção funcional (Módulo de Pedidos, Módulo de Aluguel, etc.) é implantado como um conjunto de pods. Essa separação permite que cada serviço seja escalado de forma independente para atender a demandas específicas, como um pico no processamento de pedidos (RF3) ou na busca por veículos (RF17).
- **Pods da Infraestrutura de Suporte:**
- **NATS:** Implantado no cluster para fornecer a espinha dorsal de comunicação assíncrona, essencial para notificações em tempo real (RF27) e para a confiabilidade do sistema.
- **Open Policy Agent (OPA):** Executado como um serviço para centralizar o controle de acesso e autorização (RF25), garantindo que as políticas de segurança sejam aplicadas de forma consistente em todos os microserviços.
- **Fluentd:** Geralmente implantado como um *DaemonSet* (um pod por nó) para coletar logs de todos os contêineres de forma unificada (RF26), o que é crucial para a observabilidade em um sistema distribuído.

Conceitos Transversais

Conteúdo

Esta seção descreve globalmente as principais ideias e soluções que são relevantes em várias partes do sistema, garantindo consistência na arquitetura. Para este projeto, os seguintes conceitos foram definidos:

Padrão de Arquitetura de Microsserviços: O sistema é decomposto em componentes independentes baseados em domínios de negócio, como Pedidos, Usuários, Entregas e Aluguel de Veículos.

Comunicação Assíncrona via Mensageria: O sistema utiliza o NATS como a espinha dorsal para a comunicação assíncrona entre os serviços, especialmente para a implementação de notificações em tempo real.

Controle de Acesso Centralizado: A autorização e o controle de acesso para os diferentes perfis de usuário são centralizados usando o Open Policy Agent (OPA). A lógica de políticas é escrita em uma linguagem declarativa (Rego) e desacoplada do código da aplicação.

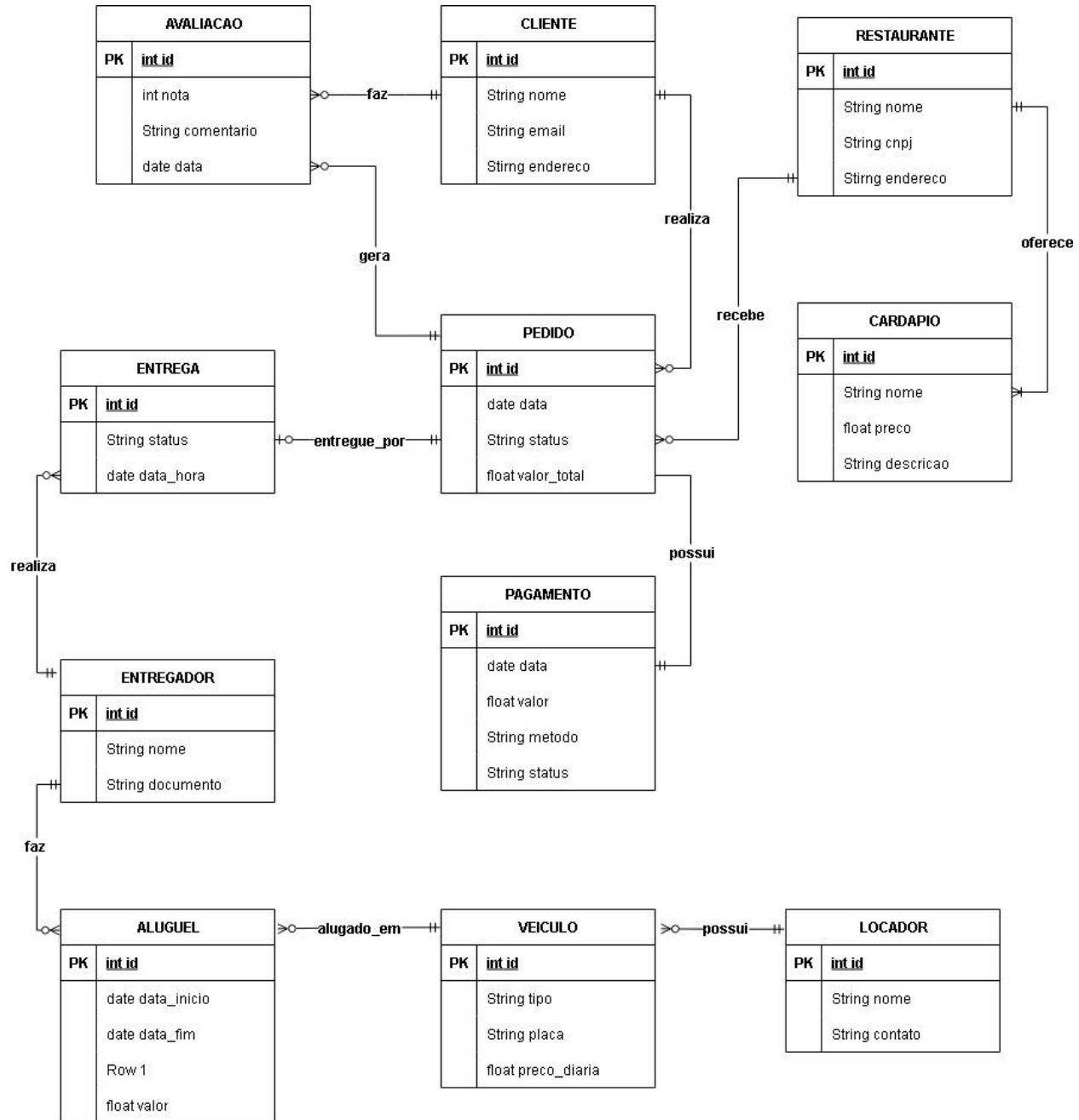
Observabilidade com Coleta de Logs Unificada: O sistema implementa um coletor de dados de logs unificado, como o Fluentd, para garantir a observabilidade em toda a arquitetura de microsserviços.

Motivação

Os conceitos transversais formam a base para a integridade conceitual e a consistência da arquitetura, sendo uma contribuição importante para atingir as qualidades internas do sistema.

- **Padrão de Microsserviços:** A escolha foi motivada pela necessidade de escalar e manter cada parte do sistema de forma independente. Isso permite, por exemplo, que uma falha no sistema de aluguel não afete o processamento de pedidos de comida, além de facilitar a manutenção.
- **Comunicação Assíncrona:** O requisito de **confiabilidade** exige que as notificações sejam entregues de forma rápida e garantida (RF27). O padrão de publicar/assinar (*pub/sub*) do NATS desacopla os serviços, melhorando a resiliência do sistema, pois uma falha temporária em um consumidor não impede que o evento seja processado posteriormente.
- **Controle de Acesso Centralizado:** Esta abordagem atende ao requisito de **segurança** (RF25) de que um usuário de um perfil não possa acessar funcionalidades de outro. Centralizar a lógica com OPA, em vez de espalhá-la pelos microsserviços, unifica o controle de acesso de forma consistente e facilita a manutenção.

- **Observabilidade:** Em uma arquitetura de microsserviços, cada serviço gera logs de forma independente. A coleta e centralização desses logs (RF26) são cruciais para a depuração, análise de problemas e para rastrear uma requisição através dos vários serviços.



Decisões Arquiteturais

Este documento detalha as decisões de arquitetura mais importantes do projeto, com o objetivo de fornecer clareza e permitir que as partes interessadas compreendam a lógica

por trás das escolhas feitas. Cada decisão é apresentada com sua motivação e a forma como será implementada.

ADR 1: Adoção de uma Arquitetura de Microsserviços e Ferramentas de Orquestração Leves

Decisão: O sistema será construído com uma arquitetura de microsserviços, dividindo-o em componentes independentes. A orquestração desses serviços será gerenciada por uma distribuição leve do Kubernetes, o **K3s**. O roteamento e o balanceamento de carga serão automatizados pelo **Traefik Proxy**, que atuará como o API Gateway.

Motivação: Essa abordagem permite que cada serviço seja implantado e escalado horizontalmente de forma independente para suportar picos de demanda, atendendo diretamente ao requisito de escalabilidade. A separação de responsabilidades aumenta a resiliência do sistema, e o uso de ferramentas leves como K3s e Traefik Proxy simplifica a complexidade operacional e a configuração de roteamento em ambientes com requisitos mínimos de recursos.

ADR 2: Uso de um Sistema de Mensageria de Alta Performance para Comunicação Assíncrona

Decisão: O sistema utilizará o **NATS** como espinha dorsal para a comunicação assíncrona entre os serviços. Ele será a ferramenta para implementar as funcionalidades de notificação em tempo real.

Motivação: O requisito de confiabilidade exige que as notificações sejam entregues de forma rápida e garantida. O padrão de comunicação de publicar/assinar (pub/sub) do NATS desacopla os produtores de eventos (ex: Serviço de Pedidos) dos consumidores (ex: Restaurante, Entregador), melhorando a resiliência do sistema. Uma falha temporária em um serviço de notificação não impede que o evento seja processado posteriormente, garantindo a entrega.

ADR 3: Implementação de um Motor de Políticas para Controle de Acesso Centralizado

Decisão: O controle de acesso e autorização para diferentes perfis de usuário será centralizado usando o **Open Policy Agent (OPA)**. A lógica de políticas será escrita em uma linguagem declarativa (Rego) e desacoplada do código da aplicação, que consultará o OPA para tomar decisões de "permitir" ou "negar".

Motivação: O requisito de segurança exige que um usuário de um perfil não possa acessar funcionalidades de outro. Em vez de espalhar a lógica de validação por todo o código dos microsserviços, o OPA permite centralizar e unificar o controle de acesso de forma elegante e consistente.

ADR 4: Estratégia de Observabilidade com Coleta de Logs Unificada

Decisão: Para garantir a observabilidade, o sistema implementará um coletor de dados de logs unificado, como o **Fluentd**.

Motivação: Em uma arquitetura de microsserviços, cada serviço gera logs de forma independente. A coleta e a centralização desses logs são cruciais para a depuração e a análise de problemas. O Fluentd pode coletar automaticamente os logs de todos os contêineres e exibi-los de forma unificada, permitindo rastrear uma requisição através dos vários serviços

Requisitos de qualidade

Esta seção documenta os requisitos de qualidade do projeto, detalhando como eles foram definidos para serem concretos e mensuráveis, influenciando diretamente as decisões arquiteturais.

- 1. Desempenho:

O sistema deve garantir um tempo de resposta rápido para operações críticas. A latência para confirmar um pedido após o pagamento não deve exceder 2 segundos em 95% das requisições durante horários de pico. O sistema precisa suportar 500 pedidos simultâneos por minuto.

- 2. Escalabilidade:

O sistema deve ser capaz de lidar com o aumento de usuários e pedidos sem comprometer o desempenho. A arquitetura precisa permitir a expansão horizontal dos serviços para atender a uma base de clientes crescente em novas cidades.

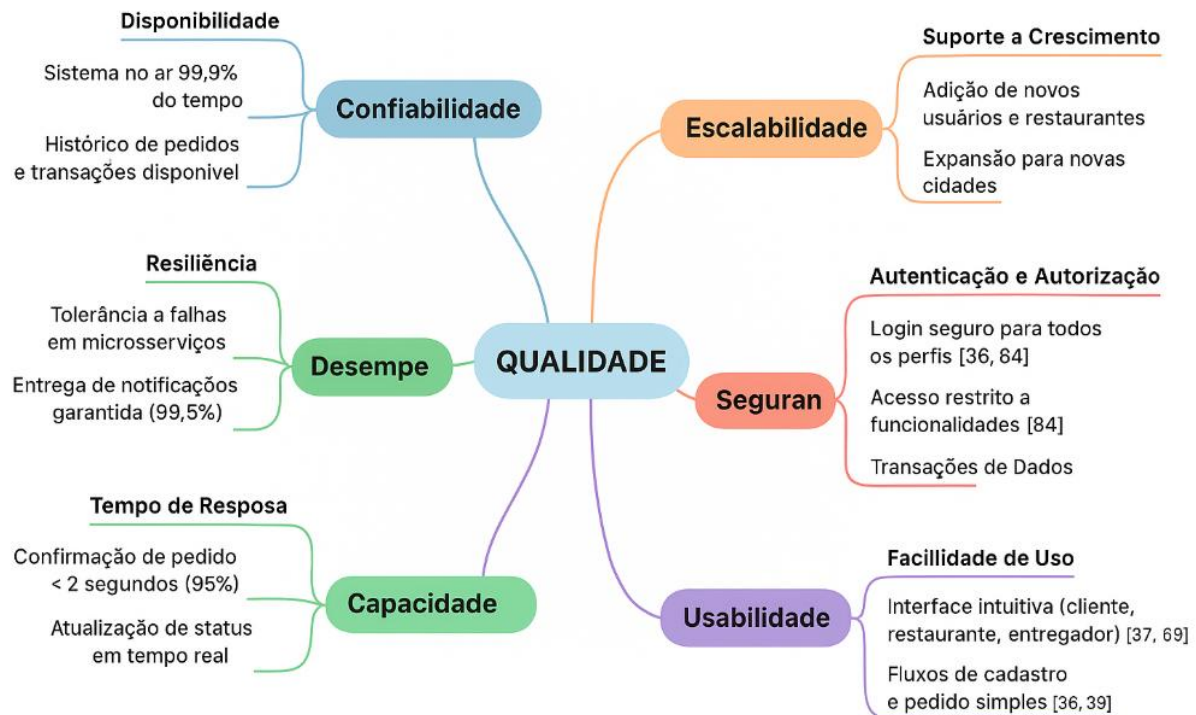
- 3. Confiabilidade:

O sistema deve ser resiliente e garantir a entrega de serviços críticos. As notificações de novos pedidos devem ser entregues a restaurantes e entregadores com uma taxa de sucesso de 99.9% em menos de 5 segundos durante o pico. O histórico de pedidos e transações deve ser armazenado de forma durável e com alta disponibilidade.

- 4. Segurança:

A plataforma deve garantir a segurança dos dados e transações. A autenticação e autorização de usuários devem ser robustas, assegurando que um usuário de um perfil não possa acessar funcionalidades exclusivas de outro.

Árvore de qualidade



Cenários de Qualidade

A seguir estão os cenários que concretizam os requisitos de qualidade do sistema, descrevendo as reações esperadas a estímulos específicos, tanto em tempo de execução (cenários de uso) quanto em situações de modificação (cenários de mudança). Estes cenários são fundamentais para guiar e avaliar a arquitetura do sistema.

Prioridade	Objetivo de Qualidade (ISO 25010)	Cenário Concreto de Qualidade
1	Usabilidade	Um novo entregador, com treinamento mínimo, deve ser capaz de se cadastrar, encontrar e alugar um veículo, e aceitar sua primeira entrega em menos de 15 minutos após abrir o aplicativo pela primeira vez. A interface deve ser clara e o fluxo de ações, intuitivo,

		para minimizar a necessidade de suporte.
2	Confiabilidade	Durante o horário de pico (19h às 21h de sexta-feira), 99.9% das notificações de novos pedidos devem ser entregues aos restaurantes e entregadores em menos de 5 segundos após a confirmação do cliente. O sistema deve manter a capacidade de processar o rastreamento de status do pedido em tempo real sem falhas.
3	Segurança	As informações de pagamento do cliente (ex: cartão de crédito) devem ser processadas através de um gateway de pagamento certificado, sem que dados sensíveis sejam armazenados de forma legível no sistema. O sistema deve garantir a correta autenticação e autorização para que um usuário de um perfil (ex: Entregador) não possa acessar funcionalidades exclusivas de outro (ex: Locador de Veículos)
4	Desempenho	Um cliente navegando pelo catálogo de restaurantes e seus cardápios deve ter os itens carregados e exibidos na tela em menos de 3 segundos, mesmo em condições de rede móvel (4G/5G).

5	Integridade & Disponibilidade	Em uma transação de aluguel de veículo, o sistema deve garantir que o veículo selecionado por um entregador seja imediatamente marcado como indisponível para outros, e a transação de pagamento entre entregador e locador seja processada de forma segura.
6	Modificabilidade (Cenário de Mudança)	Uma nova opção de pagamento digital (ex: Pix) precisa ser adicionada ao sistema. A arquitetura deve permitir que a nova funcionalidade seja desenvolvida, testada e integrada em menos de 3 semanas, com impacto mínimo nos módulos de pedido e pagamento existentes.
7	Escalabilidade (Cenário de Mudança)	A plataforma planeja expandir para uma nova cidade, triplicando o número de restaurantes e usuários ativos. A arquitetura deve suportar este aumento de carga sem degradação significativa no tempo de resposta das operações críticas (realizar pedido, aceitar entrega).

Riscos e Débitos Técnicos

A seguir, uma lista priorizada dos principais riscos e débitos técnicos identificados para a arquitetura do sistema, juntamente com as medidas propostas para mitigação.

Prioridade	Risco / Débito Técnico	Descrição	Medidas de Mitigação
1	Violação de Segurança de Dados	Dado o manuseio de informações sensíveis, como dados de pagamento de clientes e informações pessoais de múltiplos perfis de usuários, existe um alto risco de acesso não autorizado ou vazamento de dados. Uma falha na autorização poderia permitir que um tipo de usuário acessasse dados de outro (ex: Entregador acessando dados do Locador).	Implementar autenticação e autorização robustas via um serviço dedicado, utilizando padrões como OAuth 2.0. Não armazenar dados de pagamento sensíveis, delegando todo o processamento a um gateway de pagamento certificado. Realizar auditorias de segurança e testes de penetração regulares nas APIs e na infraestrutura.
2	Falha de Serviços Críticos sob Alta Carga	O sistema depende de notificações em tempo real (pedidos, status, aluguéis) e do processamento de pedidos para funcionar. O objetivo de confiabilidade exige que 99.9% das notificações de pedidos sejam entregues em menos de 5 segundos durante o horário de pico. A falha do Serviço de	Projetar os serviços críticos (Pedidos, Notificações, Aluguel) para serem escaláveis horizontalmente. Utilizar filas de mensagens para comunicação assíncrona entre os serviços, garantindo que picos de carga em um serviço não causem falhas em cascata.

		Notificações, Serviço de Pedidos ou do Banco de Dados sob alta carga representa um risco crítico, que levaria à perda de pedidos e receita.	Realizar testes de carga e estresse focados nos cenários de horário de pico para identificar e eliminar gargalos.
3	Débito Técnico: Complexidade da Integração de Domínios	A arquitetura integra dois domínios de negócios complexos: entrega de comida e aluguel de veículos. A lógica que conecta um entregador, seu aluguel de veículo ativo e uma entrega de pedido é intrinsecamente complexa. Um projeto apressado ou um acoplamento forte entre os módulos de "Pedidos" e "Aluguel de Veículos" pode gerar um débito técnico significativo, tornando futuras manutenções e evoluções lentas e propensas a erros	Isolar os domínios em microsserviços distintos (ex: Serviço de Pedidos, Módulo de Aluguel de Veículos), com comunicação via APIs bem definidas. Adotar padrões de projeto de software, como o Domain-Driven Design (DDD), para modelar e implementar a complexa lógica de negócios de forma clara e sustentável.
4	Inconsistência de Dados entre Microsserviços	Com múltiplos serviços (Pedidos, Aluguel, Pagamentos) operando de forma independente, manter a consistência dos dados torna-se um desafio. Por	Implementar padrões para transações distribuídas, como o Saga Pattern, para garantir que operações que abrangem múltiplos serviços sejam concluídas com

		exemplo, um pagamento de aluguel pode ser aprovado (no Serviço de Pagamentos), mas a atualização do status do aluguel pode falhar (no Módulo de Aluguel), levando a um estado inconsistente no sistema.	sucesso ou revertidas de forma segura em caso de falha. Utilizar uma arquitetura orientada a eventos para propagar mudanças de estado entre os serviços de forma confiável.
5	Dependência de Serviços Externos	O sistema depende de serviços de terceiros para funcionalidades essenciais, como o gateway de pagamento para processar transações e, potencialmente, serviços de mapas para otimização de rotas. A indisponibilidade ou uma mudança na API desses serviços pode impactar diretamente a funcionalidade do sistema.	Implementar o padrão de projeto Circuit Breaker para lidar com falhas temporárias nos serviços externos, evitando que a indisponibilidade deles afete todo o sistema. Desenvolver camadas de abstração para isolar o código da aplicação das APIs externas, facilitando a substituição do provedor de serviço se necessário.

Glossário

A tabela a seguir define os termos de domínio e técnicos mais importantes utilizados na documentação do sistema, garantindo um entendimento comum entre todas as partes interessadas.

Termo	Definição
Termos de Domínio e Negócio	
Cardápio	Lista de itens que um restaurante parceiro oferece para venda na plataforma.

Entregador	Usuário responsável por coletar os pedidos nos restaurantes e entregá-los aos clientes. Pode alugar veículos através da plataforma para realizar as entregas.
Inventário de Veículos	Conjunto de veículos (motos, bicicletas elétricas etc.) que um locador disponibiliza para aluguel na plataforma.
Locador de Veículos	Parceiro de negócio que cadastra e gerencia uma frota de veículos para aluguel pelos entregadores.
Status do Pedido	Etapa atual do ciclo de vida de um pedido, como "preparando", "pronto para entrega", "em rota" e "entregue".
Termos de Arquitetura e Técnicos	
API Gateway	Ponto de entrada único para todas as requisições do sistema. No projeto, este papel é desempenhado pelo Traefik Proxy, que roteia as chamadas para os microsserviços correspondentes.
Cloud-Native	Abordagem de arquitetura que utiliza tecnologias de nuvem, como contêineres e orquestração, para construir aplicações escaláveis e resilientes.
Contêiner	Unidade de software padronizada que empacota o código e todas as suas dependências. No projeto, cada microsserviço é executado em contêineres.
Fluentd	Ferramenta de código aberto utilizada como um coletor de dados para unificar, estruturar e encaminhar logs. É usada para implementar a observabilidade e o registro centralizado de logs.
K3s	Uma distribuição Kubernetes leve, empacotada como um único binário, projetada para ambientes com requisitos mínimos de recursos. É a ferramenta de orquestração escolhida para o projeto.
Microsserviços	Padrão de arquitetura que decompõe o sistema em serviços menores, independentes e baseados em domínios de negócio, como Pedidos, Aluguel e Usuários.
NATS	Sistema de mensagens de alta performance usado para a comunicação assíncrona entre os microsserviços. É a base para as notificações em tempo real usando o padrão publicar/assinar (pub/sub).
Open Policy Agent (OPA)	Motor de políticas de propósito geral que desacopla a lógica de autorização do código da aplicação. É usado para centralizar o controle de acesso, respondendo se uma ação é permitida ou negada.
Pub/Sub (Publicar/Assinar)	Padrão de comunicação assíncrona onde um serviço "publica" um evento (ex: novo_pedido) sem conhecer os "assinantes". Os serviços interessados "assinam" o evento para receber a notificação, promovendo o desacoplamento.
Rego	Linguagem declarativa utilizada para escrever as políticas de segurança e autorização que são executadas pelo OPA.
Traefik Proxy	Um "Cloud Native Edge Router" que atua como API Gateway e balanceador de carga. Ele descobre e roteia o tráfego para os serviços de forma automática, eliminando configurações manuais complexas.
Web App Responsivo	Aplicação acessível via navegador web que se adapta a diferentes tamanhos de tela (desktops, tablets e smartphones). É uma restrição de negócio do projeto, que não terá aplicativos móveis nativos.

<https://github.com/iYoNuttxD/ClickDelivery>